

UNITÉ 9 · SEMAINES 15–16

OS natif pour l'IA : agents et LLM dans le système d'exploitation

📅 Semaines 15–16 · dates à confirmer </> Laboratoire TD8 (corrigé + défi IA) ~ 4 animations

MANUEL M9 · THÉORIE DE L'UNITÉ

OS natif pour l'IA : agents et LLM dans le système d'exploitation

Lecture de base de l'unité 9 (≈ 1 h 30). Cette unité rassemble tout le cours : un agent d'IA est un processus de plus, qui utilise des appels système, des fichiers, de la mémoire, des threads et des sockets, mais dont les décisions sont prises par un modèle qui peut se tromper. **Les produits cités évoluent vite** : chaque affirmation porte sa source et sa date (vérifiées le 25/09/2026).

SOMMAIRE

- | | | | |
|----|------------------------------------|----|-------------------------------------|
| §1 | Du chat à l'agent | §4 | Les primitives d'isolation de Linux |
| §2 | Ce qu'ont annoncé Windows et Linux | §5 | Le modèle propose, l'OS dispose |
| §3 | L'inférence, une charge pour l'OS | | |

§1 Du chat à l'agent

Un modèle de langage (LLM) reçoit du texte et prédit le texte suivant. Un **agent** ajoute une boucle : le modèle propose une *action* — un appel d'outil, comme `read_file("notas.txt")` —, un programme l'exécute et le résultat revient au modèle. Un assistant peut ainsi ranger des fichiers, changer des réglages ou écrire dans une application.

Pour le système d'exploitation, c'est nouveau pour deux raisons : l'agent agit **au nom de l'utilisateur mais n'est pas l'utilisateur**, et ses décisions viennent d'un modèle statistique qui peut se tromper ou être manipulé (par exemple avec des instructions cachées dans un document qu'il lit : *injection de prompt*). C'est pourquoi les questions de cette unité sont des questions de système d'exploitation : qui est l'agent ? que peut-il toucher ? qui l'autorise ? comment l'audite-t-on ?

§2 Ce qu'ont annoncé Windows et Linux

Windows

- **Agent workspace** : un espace séparé où l'agent travaille en parallèle de l'utilisateur, avec **son propre compte** ; il n'accède aux dossiers connus (Documents, Téléchargements, Bureau,

Musique, Images, Vidéos) que si l'utilisateur l'active et donne son accord. Il s'active dans *Paramètres* → *Système* → *Composants d'IA* → *Fonctionnalités agentiques expérimentales* (préversion). Source : Microsoft Support, « Experimental agentic features ».

- **Agent connectors** : des serveurs *Model Context Protocol* (MCP) qui relient l'agent aux applications et aux outils du système, enregistrés dans l'**On-Device Registry** ; les premiers, pour l'Explorateur de fichiers et les Paramètres (Insiders, décembre 2025). Sources : Microsoft Support ; The Register, 08/12/2025.
- Principes déclarés : des actions de l'agent **observables et distinguables** de celles de l'utilisateur, l'**autorisation** de l'utilisateur pour les données et les actions, et le **moindre privilège** (jamais plus de permissions que l'utilisateur qui l'a lancé).
- **Microsoft Execution Containers** (Build 2026, 2 juin 2026) : une couche d'exécution à politiques déclaratives pour les agents sur Windows et WSL, avec une isolation par *processus*, par *session* ou dans une *micro-VM* adossée à l'hyperviseur ; en préversion précoce. Source : Windows Developer Blog, « Windows platform security for AI agents », 02/06/2026.

Linux

- Il n'y a pas un « agent du système » unique : le **noyau fournit des primitives** (§4) et les distributions et les applications les combinent.
- **Canonical (Ubuntu)** a annoncé en 2026 des fonctions d'IA centrées sur l'inférence locale avec des modèles ouverts (*inference snaps*) et des agents confinés avec les mécanismes de snap ; les premières préversions sont attendues dans Ubuntu 26.10. Il n'y aura pas d'interrupteur global de l'IA : on la désactive en supprimant les snaps. Sources : OMG! Ubuntu, 04/2026 ; Phoronix.
- Pour les accélérateurs d'IA, Linux dispose du sous-système **accele** (pilotes de NPU d'Intel et d'AMD). Dans la communauté du noyau, depuis décembre 2025, la documentation demande de déclarer l'aide d'outils d'IA avec la balise **Assisted-by** ; seule une personne peut ajouter la signature **Signed-off-by**.

Pratiquez → animation A9.1 · Architecture d'un SE agentique

§3 L'inférence, une charge pour l'OS

Pour le système d'exploitation, exécuter un modèle, c'est un programme au profil très particulier :

- **Des poids énormes et en lecture seule.** On les charge avec **mmap** (unité 6) : ils ne sont pas copiés mais lus depuis le cache de pages quand on y touche (défauts de page), et plusieurs processus partagent les mêmes pages physiques. Le micro-LLM du TD8 projette ainsi ses ≈ 0,7 Mo de poids.
- **Le cache KV.** Pour chaque token déjà traité, le modèle garde les clés et les valeurs de chaque couche pour ne pas les recalculer : une mémoire qui grandit avec la longueur de la conversation (dans le TD8, $2 \times \text{couches} \times \text{contexte} \times 64 \times 4$ octets).
- **Deux phases.** Le *prefill* traite toute la requête d'un coup ; le *decode* génère un token par étape, chacune relisant tous les poids.

- **Accélérateurs.** Les GPU et les NPU se gèrent comme des périphériques : pilotes, mémoire propre et ordonnancement de l'accès entre processus.

Pratiquez → animation A9.2 · Au cœur du micro-LLM

§4 Les primitives d'isolation de Linux

PRIMITIVE	CE QU'ELLE LIMITE	EXIGE ROOT
Utilisateurs et permissions	Les fichiers selon le propriétaire, le groupe et le mode	Pour créer des utilisateurs
Landlock (depuis 5.13)	Quels chemins du système de fichiers le processus peut ouvrir ; ports TCP (ABI 4) ; signaux et sockets abstraits hors de son périmètre (ABI 6)	Non : le processus se restreint lui-même
seccomp	Quels appels système il peut faire	Non (avec <code>no_new_privs</code>)
Namespaces	Ce qu'il voit : processus, montages, réseau, utilisateurs (la base des conteneurs)	Non, avec les namespaces utilisateur
cgroups	Combien il consomme : CPU, mémoire, E/S	En général oui (ou délégué par systemd)
<code>setrlimit</code>	Limites par processus : temps CPU, taille de fichier, descripteurs	Non, pour les abaisser

Le TD8 utilise Landlock, `setrlimit` et `alarm` dans `agent_exec` : après `landlock_restrict_self`, le processus ne peut plus sortir de l'espace de travail, même si son propre code essaie. La restriction s'hérite et ne peut pas être annulée.

§5 Le modèle propose, l'OS dispose

La leçon centrale de l'unité : **la sortie du modèle est un texte non fiable**. Le SE agentique du TD8 la traite comme n'importe quelle entrée externe :

1. **Analyseur strict** : seulement `name("arg", "arg")` d'un catalogue fermé ; tout le reste est rejeté.
2. **Politique** : chemins relatifs dans l'espace de travail, sans `..` et en résolvant les liens symboliques (`realpath`).
3. **Consentement** de l'utilisateur pour tout ce qui est destructeur.

4. **Bac à sable du noyau** (Landlock) comme dernière ligne: si la politique échoue ou est désactivée, le noyau refuse quand même l'accès.
5. **Audit** : chaque décision va dans `audit.log` avec l'uid et le pid du client (`SO_PEERCREATED`).
C'est le principe de *défense en profondeur* : aucune couche ne suppose que la précédente n'a pas échoué.

Pratiquez → animation A9.4 · Le modèle propose, le SE dispose, et TD8